

I've run into confusion since objective3.py looks like this:

```
from __future__ import annotations
```

```
from pathlib import Path
```

```
import numpy as np
```

```
import pandas as pd
```

```
from scipy import stats
```

```
import statsmodels.api as sm
```

```
import statsmodels.formula.api as smf
```

```
from utils.config import RESPONSE_VARS, ALPHA
```

Show more ▾

I see the problem.

You already modified `tweedie_glm.py` to return a dictionary:

```
return {  
    "model_fit": model_fit,  
    "lr": lr,  
    "params": params,  
    "diag": diag,  
    "fit": full,  
}
```

But `objective3.py` is still written for the old return format:

Python

Run

That line now expects **4 objects**, but `fit_tweedie_models()` returns **1 dictionary**.

What needs to change

Replace this entire block:

```
fit, lr, params, diag = fit_tweedie_models(standalone, var)

glm_fits.append(fit)
glm_lrs.append(lr)
glm_params.append(params)
glm_diag.append(diag)
```

with:

```
glm_results = fit_tweedie_models(
    standalone,
    var
)

glm_fits.append(
    glm_results["model_fit"]
)

glm_lrs.append(
    glm_results["lr"]
)

glm_params.append(
    glm_results["params"]
)

glm_diag.append(
    glm_results["diag"]
)
```

This restores compatibility with the new Tweedie output.

Next Problem

Currently you are throwing away:

Python

Run

which is the actual fitted Tweedie model.

You need to keep it.

Before the loop:

Python

Run

Add:

Python

Run

near:

```
glm_fits = []
glm_lrs = []
glm_params = []
# ...
```

Python

Run

Inside the loop add:

Python

Run

So the section becomes:

```
glm_results = fit_tweedie_models(
    standalone,
    var
```



```

    )

    glm_fits.append(
        glm_results["model_fit"]
    )

    glm_lrs.append(
        glm_results["lr"]
    )

    glm_params.append(
        glm_results["params"]
    )

    glm_diag.append(
        glm_results["diag"]
    )

    glm_models[var] = glm_results["fit"]

```

Import the New Figure Functions

At the top of objective3.py add:

```

from utils.figures import (
    plot_observed_vs_predicted,
    plot_residuals_vs_fitted,
    plot_tweedie_surface,
)

```

assuming those functions live in `utils/figures.py`.

Adjust the import path if your project structure differs.

Generate the Figures

After:

insert:

```
fit_model = glm_results["fit"]

predicted = fit_model.predict(
    standalone
)
```

Observed vs Predicted

```
plot_observed_vs_predicted(
    observed=standalone[var],
    predicted=predicted,
    response_label=var,
    out_path=(
        out_figures /
        f"{var}_observed_vs_predicted.png"
    ),
)
```

Residual vs Fitted

```
plot_residuals_vs_fitted(
    observed=standalone[var],
    predicted=predicted,
    response_label=var,
    out_path=(
        out_figures /
        f"{var}_residuals_vs_fitted.png"
    ),
)
```

Tweedie Surface

```
plot_tweedie_surface(
    glm_fit=fit_model,
    setup_label="Standalone",
    environments=sorted(
        standalone["Environment"].unique()
    ),
    response_label=var,
    out_path=(
        out_figures /
        f"{var}_tweedie_surface.png"
    ),
)
```

Why This Location?

This is the correct place because:

```
objective3.py
├─ fits Tweedie GLM
├─ computes LR tests
├─ computes coefficients
├─ computes diagnostics
└─ should generate Tweedie figures
```

The figures depend on:

Python

Run

which only exists after the model is fit.

Putting figure generation in `main.py` would require passing around fitted model objects and complicating the architecture.

One More Statistical Issue

Your current model is:

```
full_formula =
```

Python

Run

and:

```
standalone["SPL_dB"] =
```

Python

Run

This means:

Python

Run

treats SPL as a factor.

The resulting surface will look like:

```
step
step
step
step
```

rather than:

```
smooth curve
```

because each SPL level gets its own coefficient.

If your research objective is to model the physical relationship between SPL and harvested power, I would strongly recommend:

```
full_formula =
f"{variable} ~ C(Environment) * SPL_dB"
```

Python

Run

and remove:

```
standalone["SPL_dB"] =
```

Python

Run

This keeps SPL continuous, which is generally more appropriate for a response-surface visualization and for a sound-pressure-to-power relationship. It will also make the Tweedie surface plots much more informative.



Where did you find this block?

```
fit, lr, params, diag = fit_tweedie_models(standalone, var)
```

```
glm_fits.append(fit)
glm_lrs.append(lr)
glm_params.append(params)
glm_diag.append(diag)
```

In the `objective3.py` code you pasted, it's here inside the `for var in RESPONSE_VARS:` loop:

```
fit, lr, params, diag = fit_tweedie_models(standalone, var)
glm_fits.append(fit)
glm_lrs.append(lr)
glm_params.append(params)
glm_diag.append(diag)
```

Specifically, it appears immediately after:

```

16:         posthoc_spl = posthoc_spl.assign(variable=var, factor="SPL_dB")

```

Python

Run

and immediately before:

Python

Run

So the relevant section of your pasted code is:

```

if isinstance(posthoc_spl, pd.DataFrame):
    kw_posthoc_tables.append(posthoc_spl.assign(variable=var, factor="SPL_dB"))
if isinstance(posthoc_env, pd.DataFrame):
    kw_posthoc_tables.append(posthoc_env.assign(variable=var, factor="Environment"))

fit, lr, params, diag = fit_tweedie_models(standalone, var)
glm_fits.append(fit)
glm_lrs.append(lr)
glm_params.append(params)
glm_diag.append(diag)

omnibus_rows.append(omnibus_factorial_table(df, var))

```

However, this reveals something important:

Your pasted files are inconsistent

The `tweedie_glm.py` you pasted currently ends with:

```

return {
    "model_fit": model_fit,
    "lr": lr,
    "params": params,
    "diag": diag,
    "fit": full,
}

```

while the `objective3.py` you pasted still expects:

Python

Run

Those two versions cannot both be running successfully at the same time.

One of three things is true:

1. You modified `tweedie_glm.py` but haven't run the code yet.
2. `objective3.py` is currently crashing with:

Python

Run

3. The code you actually executed is a different copy than the one you pasted.

Before making any more changes, I would check:

Bash

and see whether Objective 3 currently runs successfully.

If it does run successfully, then the version of `tweedie_glm.py` that Python is actually importing is not the same version you've shown here.

At this point, the safest next step is for you to show me the **current traceback/error message** (if any), or tell me whether `main.py` currently completes successfully. That will determine whether we need to fix a return-value mismatch first before adding any figure-generation code.



For the this suggestion:

```
full_formula =  
f"{variable} ~ C(Environment) * SPL_dB"  
  
main_formula =  
f"{variable} ~ C(Environment) + SPL_dB"
```

The variable is undefined says the interpreter.

That error is occurring because I was showing the **replacement string contents** for the formulas inside `fit_tweedie_models()`, not standalone code.

Inside your function, `variable` is a parameter:

Python

Run

Therefore this is valid **inside that function**:

```
full_formula = f"{variable} ~ C(Environment) * SPL_dB"  
main_formula = f"{variable} ~ C(Environment) + SPL_dB"
```

Python

Run

But if you paste:

Python

Run

outside the function (for example at module level), Python correctly complains that:

```
NameError: variable is not defined
```

because `variable` only exists inside:

Python

Run

What you should actually modify

In your current `tweedie_glm.py`, locate:


```
full_formula = f"{variable} ~ C(Environment) * C(SPL_dB)"  
main_formula = f"{variable} ~ C(Environment) + C(SPL_dB)"
```

Python

Run

and replace only those lines with:

```
full_formula = f"{variable} ~ C(Environment) * SPL_dB"  
main_formula = f"{variable} ~ C(Environment) + SPL_dB"
```

Python

Run

leaving them inside the function.

Why make this change?

Currently:

Python

Run

treats SPL as a categorical factor:

```
60 dB -> coefficient 1  
70 dB -> coefficient 2  
80 dB -> coefficient 3  
...
```

This is appropriate if your goal is:

Compare discrete SPL treatment groups.

However, your Objective 1 already analyzes thresholds and correlations, and your Tweedie GLM appears intended to model the physical response.

In that case:

Python

Run

```
SPL_dB
```

treats SPL as continuous:

```
60, 61, 62, ... 120
```

which lets the Tweedie-log model estimate a smooth response curve.

For a harvested-power system, that is usually more scientifically meaningful.

One more thing

Your `objective3.py` currently contains:

Python

Run

```
standalone["SPL_dB"] = standalone["SPL_dB"].astype(int)
```

This line is harmless if SPL remains continuous. It only converts:

```
60.0 -> 60  
70.0 -> 70
```

You can leave it there.

The crucial change is removing:

Python

Run

```
C(...)
```

around `SPL_dB` in the Tweedie formulas.

